

35.16. Un avion qui vole : AVION dans son fichier, la croix pour le piloter

AVION, le dessin tout fait de la bibliothèque, rangé dans perso_AVION.cpp ; sprite16() le pose, la croix le fait voler d'un pixel par image, sans sortir de l'écran.

1. Ce qu'il faut comprendre

Ce qui est nouveau ici : un personnage qu'on pilote. L'avion est rangé comme BONHOMME : son dessin dans perso_AVION.cpp, la liste personnages.cpp qui le verse, et principal.cpp qui verse la liste d'une ligne. Le dessin est celui de la bibliothèque (« Les tuiles », onglet des modèles, AVION).

Sa place, en pixels : px (de gauche à droite) et py (de haut en bas). Il part de (72, 64), le milieu de l'écran. À chaque image, sprite16(0, px, py, AVION) le pose à sa place : les lutins 0 à 3, en carré.

La croix : bouton(DROITE) vaut vrai tant que la flèche droite est enfoncée ; alors px++ — l'avion avance d'un pixel par image, soit 60 pixels par seconde. Même chose pour GAUCHE (px--), BAS (py++) et HAUT (py--). Deux flèches à la fois : il vole en diagonale.

Les bords de l'écran : l'écran fait 160 × 144 pixels, et l'avion 16 × 16. Son coin haut-gauche va donc de 0 à 144 (160 – 16) en largeur, et de 0 à 128 (144 – 16) en hauteur. Les conditions px < 144, px > 0, py < 128, py > 0 le gardent entier à l'écran.

Pourquoi au pixel et pas à la case : un lutin se pose au pixel près — le vol est fluide. Une tuile du fond, elle, saute de 8 pixels à chaque pas.

À toi : fais-le voler plus vite — px = px + 2 ; — ou ajoute un deuxième avion, sprite16(4, ..., AVION), avec les lutins 4 à 7.

Les fonctions de cette leçon — et le tuto de chacune

- #include <sprite16> → [35.2. La fonction sprite16\(\) — un personnage de 16 × 16](#)
- #include <bouton> → [0.65.5. La fonction bouton\(\) — lire la manette](#)
- #include <Perso> → [35.1. Perso — dessiner des personnages](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **469 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

```
Gravé dans la cartouche :
démarrage      55 o ← la console, à l'allumage – la mise en route de la console
main()         194 o ← démarrage – ta fonction
VBlank         8 o  ← la console, à chaque image – compte les images, 60 fois par seconde
AttendreImage  28 o ← image() – attend l'image suivante
AttendreVBlank 16 o ← démarrage – attend que l'écran soit libre
RangerLutins   11 o ← démarrage – range les 40 lutins hors de l'écran
```

```

InstallerTransfert  15 o ← démarrage – installe la copie des lutins en mémoire rapide
CopierTuiles        31 o ← démarrage – copie les lettres et les dessins en mémoire vidéo
LireManette         37 o ← bouton() – lit les huit boutons
Tuiles              64 o ← CopierTuiles – 0 lettre sur 43, et 4 tuiles dessinées
DonneesTransfert    10 o ← InstallerTransfert – la petite routine qui copie les lutins

```

Lettres gravées : aucune

Pas gravé (jamais appelé) : AvancerAirs, AttendreAcces, AttendreFinVBlank, AdresseCase, EffacerCases, EcrireTexte, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, EffacerCarte, Hasard, EffacerFond, EffacerPanneau, Notes

Fonctions du langage appelées :

```

sprite()   x 4   104 o sur place, dans main()
bouton()   x 4    20 o sur place, dans main() + LireManette (37 o)
image()    x 1     6 o sur place, dans main() + AttendreImage (28 o)
sprite16() x 1     0 o sur place, dans main()

```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```

// L'avion vole : la croix le déplace d'un pixel par image.

#include "personnages.cpp" // la liste : AVION
#include <sprite16>        // sprite16() : quatre lutins en carré
#include <bouton>           // bouton() : lit un bouton de la manette

```

2. UNE VARIABLE — PX

```
uint8_t px = 72; // la place de l'avion, en pixels : de 0 à 144 (160 - 16)...
```

3. UNE VARIABLE — PY

```
uint8_t py = 64; // ...et de 0 à 128 (144 - 16). Il part du milieu.
```

4. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```

int main() {
    while (true) {
        image();

        if (bouton(DROITE) && px < 144) px++; // un pixel par image
        if (bouton(GAUCHE) && px > 0)  px--;
        if (bouton(BAS) && py < 128)  py++;
        if (bouton(HAUT) && py > 0)   py--;

        sprite16(0, px, py, AVION); // l'avion à sa place : lutins 0 à 3
    }
}

```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



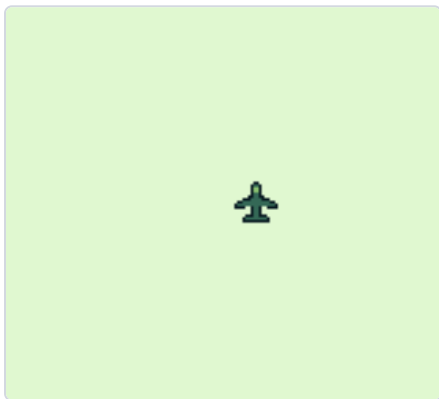
la toute première image — la console vient de démarrer — image 1



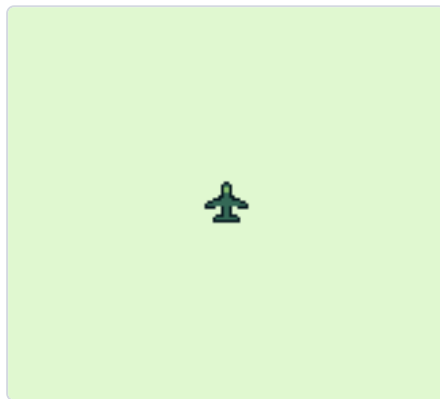
un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce que les touches changent

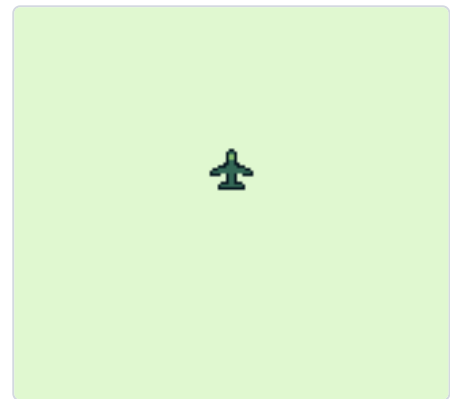
Chaque bouton que le programme lit a été enfoncé douze images, puis relâché. Voici l'écran juste après.



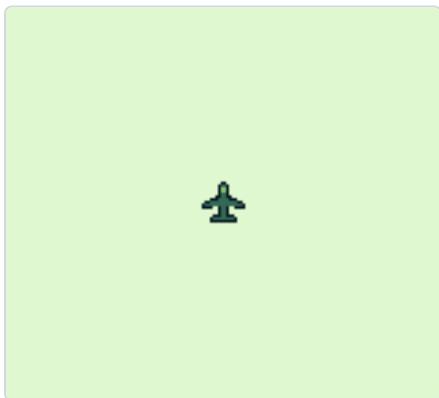
DROITE — l'écran a changé



GAUCHE — l'écran a changé



HAUT — l'écran a changé



BAS — l'écran a changé

5. Ce qu'on doit voir

L'avion au milieu de l'écran ; avec les flèches, il vole dans les quatre directions, sans jamais sortir de l'écran.

6. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ l'avion part du milieu, en (72, 64) (72, 64)

✓ la flèche droite le pousse à droite (px = 92)

✓ la flèche haut le fait monter (py = 44)

✓ à gauche, il s'arrête au bord : px = 0

✓ le lutin suit sa place

Ensuite : **35.17. La vitesse de l'avion : une variable, vitesse** — chapitre 6 : Dessiner. · gameboy3 — écrire une cartouche Game Boy en C++